

LLM Context Compaction Proxy

技术文档

LLM Context Compaction Proxy Contributors

2026 年 8 月

目录

1	概述	3
1.1	工作原理	3
1.2	环境要求	3
2	架构设计	3
2.1	请求流程	3
2.2	模块划分	4
3	压缩技术栈	4
3.1	V3 核心技术 (8 项)	4
3.2	V4 增强技术 (10 项)	5
4	配置说明	6
4.1	核心配置	6
4.2	压缩行为	6
4.3	双引擎压缩	6
4.4	安全	6
4.5	会话与记忆	6
5	接入指南	7
5.1	DeepSeek	7
5.2	DeepSeek Harness	7
5.3	Hermes Agent	8
5.4	Claude Code	8
5.5	OpenClaw	9
6	原有上下文系统屏蔽	9
6.1	屏蔽原则	9
6.2	OpenClaw	10

6.3	Hermes Agent	10
6.4	DeepSeek Harness	10
6.5	Claude Code	11
6.6	屏蔽效果验证	11
7	安全与可靠性	11
7.1	认证	11
7.2	密钥保护	12
7.3	并发安全	12
7.4	可靠性组件	12
8	验证结果	12
9	常见问题	12
9.1	401 Unauthorized	12
9.2	上下文未压缩	13
9.3	格式错误	13

1 概述

LLM Context Compaction Proxy 是一个部署在 AI Agent 与 LLM API 之间的透明上下文压缩代理。当对话上下文逼近模型上下文窗口上限时, 它自动压缩历史消息并重试, 使 Agent 能够长时间运行而不触碰上下文窗口墙。

代理累计实现 **33 项压缩技术**, 横跨 7 代演进 (V3 核心、V4 增强、V5 会话、V6 Provider、V7 MemSkill), 技术来源涵盖学术论文 (ARC、AFM、PACMS、CoMem、MemSkill) 与生产实践 (Cursor、Devin、SWE-agent、Hermes、Claude Code)。

1.1 工作原理

AI Agent	Compaction Proxy	LLM Provider
(Claude/GPT)	localhost:8198	(any API)

- Monitor context %
- Auto-compress
- Preserve key info
- Route to provider

1. **透传 (Passthrough)**——上下文未达阈值时请求零开销直达上游;
2. **检测 (Detect)**——上下文达到阈值 (默认 80%) 时触发压缩;
3. **压缩 (Compress)**——使用独立的压缩模型对旧消息生成摘要;
4. **恢复 (Resume)**——压缩结果替换旧消息, Agent 无缝继续。

1.2 环境要求

- Python 3.10+
- aiohttp (pip install aiohttp)
- 上游 LLM API (OpenAI / Anthropic / Gemini / DeepSeek / Ollama 等)

2 架构设计

2.1 请求流程

Request Flow:

Agent → Proxy → [context check] → Upstream

if > 80% full:

Compact Compaction Model (separate provider)
Engine Compressed Summary

Replace old messages
with compressed summary

→ Upstream (with compacted context)

2.2 模块划分

- **Provider 抽象层**: OpenAI / Anthropic / Gemini 三大 API 格式的统一适配, 自动探测请求来源与上游格式;
- **压缩引擎**: 可插拔架构, 内置默认引擎与双引擎 (Dual-Layer);
- **记忆系统**: 语义记忆 (SemanticMemory)、跨会话存储 (SessionStore, SQLite+FTS5)、用户画像 (UserProfile);
- **技能系统 (V7)**: 技能注册表 (SkillRegistry)、技能控制器 (SkillController)、自演进记忆技能 (MemSkill);
- **可靠性组件**: 熔断器 (CircuitBreaker)、抖动检测 (ThrashingDetector)、压缩缓存 (CompactionCache)、指标系统 (Metrics)。

3 压缩技术栈

3.1 V3 核心技术 (8 项)

#	技术	说明
1	ARC 地址化引用	以紧凑的 @ref 指针替换重复内容
2	AFM 自适应保真	消息级 Full/Compressed/Placeholder 三级保真
3	PACMS 子模选择	贪心、保真感知的消息选择
4	CCL 承诺提取	提取并保留对话中的承诺
5	抖动检测	检测压缩循环 (3 次/5 条消息)
6	两阶段 Token 估算	CJK 感知的精确计数
7	预飞安全验证	发送前验证上下文合法性
8	并行压缩	大上下文分块并行压缩

3.2 V4 增强技术 (10 项)

#	技术	来源
9	情节—语义双层记忆	arXiv:2605.17625
10	思考掩码	Kevin-32B / Devin 模式
11	标签选择性保留	SWE-agent / memor-ai
12	结构感知工具输出压缩	memor-ai
13	密钥脱敏	Cursor / memor-ai
14	缓存优化消息排序	层哈希感知 + <code>cache_control</code> 断点
15	增量压缩	CoMem (arXiv:2605.30842)
16	四信号记忆评分	语义 50%、近因 25%、类型 15%、质量 10%
17	查询位置优化	查询置尾、配对邻接、近期窗口
18	压缩项剪枝	OpenAI 模式

4 配置说明

所有配置均通过环境变量提供, 默认值见下表。

4.1 核心配置

变量	默认值	说明
COMPACTION_PROXY_PORT	8198	代理监听端口
COMPACTION_PROXY_UPSTREAM	http://localhost:11434/v1	上游 LLM API 基地址
COMPACTION_PROXY_UPSTREAM_IS_ANTHROPIC	auto	强制 Anthropic 格式 (true/false)
COMPACTION_PROXY_MODEL	gpt-4o-mini	压缩/摘要模型

4.2 压缩行为

变量	默认值	说明
COMPACTION_PROXY_KEEP_TURNS	6	保留的最近轮次 (不压缩)
COMPACTION_PROXY_MAX_RETRIES	3	压缩 API 最大重试次数
COMPACTION_PROXY_TIMEOUT	120	压缩超时 (秒)
COMPACTION_PROXY_PARALLEL_BLOCKS	3	并行压缩块数
COMPACTION_PROXY_PREEMPTIVE_THRESHOLD	0.80	上下文 80% 时预压缩

4.3 双引擎压缩

变量	默认值	说明
COMPACTION_PROXY_ENGINE	default	压缩引擎 (default/dual-layer)
COMPACTION_PROXY_DUAL_GATEWAY_RATIO	0.15	L1 网关: 保留 15%(85% 缩减)
COMPACTION_PROXY_DUAL_AGENT_RATIO	0.50	L2 Agent: 保留 L1 输出的 50%

4.4 安全

变量	默认值	说明
COMPACTION_PROXY_API_SECRET	空	代理访问密钥; 为空时仅放行回环来源
COMPACTION_PROXY_REDACT_SECRETS	1	自动脱敏消息中的密钥

4.5 会话与记忆

变量	默认值	说明
----	-----	----

COMPACTION_PROXY_BACKGROUND_SESSIONS	3	并行压缩的后台会话数
COMPACTION_PROXY_LLM_MEMORY	1	启用 LLM 记忆提取
COMPACTION_PROXY_MEMSKILL	0	启用自演进 MemSkill

5 接入指南

代理在 localhost:8198 监听, 兼容 OpenAI / Anthropic / Gemini 三种 API 格式。以下各节给出已实测验证的接入方式。

5.1 DeepSeek

DeepSeek 使用 OpenAI 兼容格式, 直接将其作为上游即可:

```
export COMPACTION_PROXY_UPSTREAM=https://api.deepseek.com/v1
export COMPACTION_PROXY_MODEL=deepseek-chat
python3 compaction-proxy.py
```

代理内置 DeepSeek 模型上下文窗口注册表 (deepseek-chat / deepseek-coder / deepseek-r1 / deepseek-v3, 均为 128K), 自动完成上下文压力预估。客户端侧只需将 base URL 指向代理:

```
export OPENAI_BASE_URL=http://localhost:8198/v1
export OPENAI_API_KEY=your-key
```

5.2 DeepSeek Harness

DeepSeek Harness(官方文档 <https://www.deepseek.com/harness/en/>) 是 DeepSeek 官方的 Agent 框架 (developer preview), 基于 Cordis 插件系统构建——模型、工具、技能、会话、沙箱、存储等一切能力均为可替换插件。其模型层支持 openai-completions(OpenAI 兼容) 与 Anthropic 两种协议, 可通过自定义 provider 将 baseURL 指向任意自托管端点。

本代理即以此方式接入。本机配置位于 \$DSH_HOME/settings.yaml (默认 ~/.dsh/settings.yaml):

```
# ~/.dsh/settings.yaml
llm-pi-ai:
  providers:
    xfyun-maas:
      displayName: 讯飞 MaaS via V6 Proxy
      apiKeyEnv: XF_MASS_API_KEY          # 凭证走环境变量引用, 不落盘
      api: openai-completions             # OpenAI 兼容协议
      baseURL: http://127.0.0.1:8198      # 指向压缩代理
  models:
    - id: xsparkx2agent
      name: xsparkx2agent
      contextWindow: 131072
      maxTokens: 8192
```

配置要点:

- `baseUrl` 指向本代理 `http://127.0.0.1:8198`, 所有模型请求先经过代理的上下文监测与压缩层;
- `api: openai-completions` 与代理 `/v1/chat/completions` 端点匹配; 模型发现使用 OpenAI 兼容 `GET /models`;
- 凭证以环境变量引用 (`apiKeyEnv`), 密钥不写入配置文件;
- 模型变更即时生效, 无需重启 Harness 服务。

已验证: 本机 Harness 的 `xfyun-maas` provider 已指向 `http://127.0.0.1:8198`, 代理 `/v1/chat/completions` 端点路由正常。

5.3 Hermes Agent

Hermes Agent 在 `7.hermes/config.yaml` 的 `model` 段中配置模型端点。将其 `base_url` 指向本代理即可获得自动压缩能力:

```
# ~/.hermes/config.yaml
model:
  api_mode: chat_completions      # OpenAI 格式
  base_url: http://127.0.0.1:8198/v1 # 指向压缩代理
  default: xopglm51               # 或任意上游可用模型
  provider: memory-proxy          # 保留原 provider 标识
```

代理的 `/v1/chat/completions` 端点与 Hermes 的 `chat_completions` 模式完全兼容 (已验证), 并支持 Anthropic 格式的 `/v1/messages` 端点。

5.4 Claude Code

Claude Code 原生使用 Anthropic Messages API。通过环境变量或 `7.claude/settings.json` 将 API 基地址指向代理:

```
# 方式一: 环境变量
export ANTHROPIC_BASE_URL=http://localhost:8198
export ANTHROPIC_AUTH_TOKEN=<your-token>
claude
```

```
// 方式二: ~/.claude/settings.json
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:8198",
    "ANTHROPIC_MODEL": "claude-sonnet-5"
  }
}
```


已验证:POST /v1/messages 路由存在且 Anthropic 格式识别成功 (无密钥请求返回 401, 格式错误则返回 400/404, 可据此区分)。

5.5 OpenClaw

OpenClaw 通过 openclaw.json 的 models.providers 配置模型网关。将 provider 的 baseUrl 指向本代理即可让全部 Agent 流量经过压缩层:

```
{
  "models": {
    "providers": {
      "compaction-proxy": {
        "baseUrl": "http://127.0.0.1:8198",
        "apiKey": "<redacted>",
        "api": "openai-completions",
        "models": [{ "id": "xopdeepseekv4flash" }]
      }
    }
  },
  "plugins": {
    "entries": {
      "v6-compaction-provider": {
        "enabled": true,
        "config": {
          "proxyUrl": "http://127.0.0.1:8198",
          "model": "xsparkx2agent"
        }
      }
    }
  }
}
```

已验证: 本机 OpenClaw 的 xunfei provider 已直接指向 http://127.0.0.1:8198, 且 v6-compaction-provider 插件经 /summarize 端点接入压缩引擎 (即 OpenClaw 上下文压缩链路已全程经由本代理)。

6 原有上下文系统屏蔽

接入本代理后, 为避免多个压缩层互相冲突 (双重压缩、重复刷记忆), 应屏蔽各接入方自带的上下文管理系统, 使压缩完全由本代理统一接管。

6.1 屏蔽原则

- **单一压缩源:** 只有本代理执行上下文压缩, 接入方自身不再触发压缩;
- **手动压缩保留:** 部分接入方保留手动压缩命令 (如 /compact), 便于必要时人工触发;

- **可回滚**: 所有修改均保留原配置备份。

6.2 OpenClaw

修改 `openclaw.json` 中 `agents.defaults.compaction` 段:

```
"compaction": {
  "enabled": false,           // 关闭原版触发+替换+内置压缩
  "midTurnPrecheck": { "enabled": false },
  "memoryFlush": { "enabled": false },
  "qualityGuard": { "enabled": false },
  "provider": "v6-proxy"     // 保留 V6 插件作为压缩提供方
}
```

- 修改即时生效 (gateway 热重载, 无需重启);
- 自动压缩关闭后, 超长上下文完全交由本代理 (127.0.0.1:8198) 在 LLM 网关层处理;
- 手动 `/compact` 仍可用, 走 `v6-proxy` 压缩链路。

6.3 Hermes Agent

修改 `7.hermes/config.yaml`:

```
compression:
  enabled: false # 关闭 Hermes 自带自动压缩
```

- `compression.enabled: false` 会禁用 Hermes 全部自动压缩 (源码 `native_compaction.py` 明确注释);
- 手动 `/compress` 命令仍可触发压缩;
- 配置修改后需重启 Hermes 生效。

6.4 DeepSeek Harness

DeepSeek Harness 基于 Cordis 插件系统, 自带压缩后端插件 `dsh-compaction-basic` (Token 计量驱动, 默认 `auto: true` 注册步骤边界压力检测与溢出恢复监听)。通过 profile 的 `cordis.patch.yml` 将其关闭:

```
# ~/.dsh/profiles/web/cordis.patch.yml (headless 同理)
- id: dsh-compaction-basic
  config:
    auto: false # 关闭 Harness 原生自动压缩
```

- `auto: false` 后, Harness 不再自行在步骤边界压缩或做溢出恢复, 超长上下文完全交由本代理在 LLM 网关层处理;

- 手动压缩命令仍可用;
- 修改需重启 dsh 进程生效;
- 模型接入经 \$DSH_HOME/settings.yaml 指向代理 (baseUrl: http://127.0.0.1:8198), 与屏蔽配置互不冲突。

6.5 Claude Code

修改 7.claude/settings.json:

```
{
  "autoCompactEnabled": false,    // 关闭 Claude Code 自动压缩
  "autoCompactWindow": 110000
}
```

- autoCompactEnabled: false 后,Claude Code 不再自行压缩上下文;
- 压缩由指向的代理 (如 ANTHROPIC_BASE_URL) 接管;
- 手动压缩指令仍可用。

6.6 屏蔽效果验证

接入方	原系统	屏蔽后状态
OpenClaw	compaction 四项开关	全部 false, 代理接管
Hermes	compression.enabled	false, 代理接管
Claude Code	autoCompactEnabled	false, 代理接管

屏蔽完成后, 通过 GET /health 确认代理运行正常 (circuit_breaker.state = closed), 三方请求均经 8198 端口由本代理统一压缩。

7 安全与可靠性

7.1 认证

- 设置 COMPACTION_PROXY_API_SECRET 后, 所有敏感端点 (/summarize、/compact、/sessions/*、/skills/*、/profile、/memory、/arc/*) 要求 Authorization: Bearer <secret> 或 X-API-Key: <secret>;
- 未设置密钥时, 仅放行回环来源 (127.0.0.1 / ::1), 非回环客户端一律返回 401(纵深防御);
- 核心转发端点 (/chat/completions、/v1/messages) 保持无代理级认证, 依赖上游密钥校验, 以兼容本机 Agent 集成。

7.2 密钥保护

- Gemini API key 通过 `x-goog-api-key` 请求头传递, 不进入 URL 查询参数, 避免泄露至访问日志;
- 支持自动脱敏消息中的密钥 (`COMPACTION_PROXY_REDACT_SECRETS=1`)。

7.3 并发安全

- 压缩系统提示词以参数传递 (`system_prompt_override`), 不再修改模块级全局变量, 消除并发请求间的提示词竞态;
- `SemanticMemory` 读写由线程锁保护, 落盘采用原子替换;
- 压缩缓存按消息内容与自定义指令双重键控, 避免不同指令串用缓存;
- `_memskill_selected` 等内部字段在请求入口即被消费移除, 不会残留至上游请求体。

7.4 可靠性组件

- **熔断器**: 3 次连续失败后进入 60 秒冷却;
- **抖动检测**: 连续快速压缩 (3 次/5 条消息) 时切换为激进截断;
- **压缩安全验证**: 压缩结果 token 数超过原文时降级为截断, 防止膨胀;
- **压缩缓存**: 30 分钟 TTL, 避免对未变化上下文重复压缩。

8 验证结果

测试项	输入	结果
<code>/health</code>	GET	HTTP 200, 熔断器 closed
<code>/summarize</code>	2 条消息	HTTP 200, 0.0s, 返回有效摘要
<code>/summarize</code>	12 轮 (24 条)	HTTP 200, 13.1s, 1948 字符, 保留轮次编号
并发压缩	6 路不同指令	全部 200, 无提示词串用
异常路径	缺 key / 坏 JSON / 空 体	401 / 400 / 200(符合预期)
<code>/compact</code>	16 条, keep=3	200, 16 → 6 条
认证端点	本机回环	全部 200(回环放行)

9 常见问题

9.1 401 Unauthorized

- 检查代理自身密钥 (`COMPACTION_PROXY_API_SECRET`);

- Anthropic 格式使用 `x-api-key` 头,OpenAI 格式使用 `Authorization: Bearer <key>` 头。

9.2 上下文未压缩

- 检查 `COMPACTION_PROXY_PREEMPTIVE_THRESHOLD`(默认 0.80);
- 确认压缩模型在上游可用;
- 查看日志 `7.local/share/compaction-proxy/proxy.log`。

9.3 格式错误

- Anthropic 客户端: 使用 `/v1/messages`;
- OpenAI 兼容客户端: 使用 `/v1/chat/completions`;
- 代理自动探测格式, 但显式端点可避免歧义。

附: 环境变量速查

完整配置模板见 `.env.example`。常用变量:

- `COMPACTION_PROXY_PORT`——监听端口 (默认 8198);
- `COMPACTION_PROXY_UPSTREAM`——上游 API 基地址;
- `COMPACTION_PROXY_MODEL`——压缩模型;
- `COMPACTION_PROXY_API_SECRET`——代理访问密钥;
- `COMPACTION_PROXY_ENGINE`——压缩引擎 (default / dual-layer);
- `COMPACTION_PROXY_MEMSKILL`——启用 MemSkill(0/1)。